Object-Oriented Programming System (OOPS)

- Object-Oriented Programming System (OOPS) is a programming approach that focuses on creating **objects** which represent real-world entities.
- In OOPS, a program is divided into **classes**, and classes are used to create **objects**.
- Each object contains **data (variables)** and **behavior (methods)**. OOPS makes programs easy to understand, reuse, modify, and maintain. It is widely used in real-world software development because it improves security and scalability.

Example:

- Class → Student
- Object → Ravi, Anita
- Attributes → name, age, marks
- Methods → study(), write_exam()

What is Class?

A **class** is a logical entity that defines:

- Variables (data)
- Functions (methods)

A **class** is a blueprint or template that defines properties and methods of an object.
A class does not occupy memory until an object is created.

What is Object ?

- An **object** is a real instance of a class that occupies memory.
- Multiple objects can be created from a single class.
- Each object has its own data but shares the same class structure.

Example Code:

```python
class Student:
    pass

s1 = Student()
s2 = Student()
```

Constructor (__init__()) :

- A constructor is a special method that is automatically executed when an object is created.
- The constructor is mainly used to initialize object data.
- In Python, the constructor method is named __init__().
- It helps assign initial values to instance variables.
- Each object has its own constructor call.

Example Code:

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

---

self Keyword :

- The self keyword represents the current object of the class.
- It is used to access instance variables and methods inside the class.
- Without self, Python cannot differentiate between local variables and object variables.
- Python automatically passes self during method calls.
- Using self improves clarity and object reference.

Example Code:

```python
class Demo:
    def show(self):
        print("Hello")
```

Variables in OOPS :

- Instance variables are variables that belong to a specific object.
- Each object has its own copy of instance variables.
- Class variables are shared among all objects of the class.
- Class variables are defined outside the constructor.
- Instance variables are defined using self.

Example Code:

```python
class Student:
    college = "ABC College"

    def __init__(self, name):
        self.name = name
```

Methods in Class :

- Methods are functions that are defined inside a class and are used to describe the behavior of an object.
- Methods allow objects to perform actions and operate on data stored inside the class.
- In Python, methods are mainly classified into instance methods, class methods, and static methods.

Instance Method :

- An instance method is a method that works with object data and uses the self keyword.
- It can access both instance variables and class variables of the class.
- Instance methods are the most commonly used methods in object-oriented programming.
- These methods are called using an object of the class.

Example Code:

```python
class Student:
    def __init__(self, name, marks):
        self.name = name            # instance variable
        self.marks = marks          # instance variable

    def display(self):              # instance method
        print("Name:", self.name)
        print("Marks:", self.marks)


s1 = Student("Ravi", 85)
s1.display()
```

Class Method :

A class method is a method that works with class-level data and uses the cls keyword.
It is defined using the @classmethod decorator.
Class methods are mainly used to modify or access class variables.
These methods can be called using either the class name or an object.

Example Code:

```python
class College:
    college_name = "ABC Engineering College"   # class variable

    @classmethod
    def show_college(cls):                     # class method
        print("College Name:", cls.college_name)


College.show_college()
```

Static Method :

- A static method is a method that does not depend on object data or class data.
- It does not use self or cls as parameters.
- Static methods are defined using the @staticmethod decorator.
- They are used to perform general utility operations related to the class.

Example Code:

```python
class MathOperations:
    @staticmethod
    def add(a, b):                  # static method
        print("Sum:", a + b)


MathOperations.add(10, 20)
```

Combined Example:

```python
class Employee:
    company = "TechSoft"          # class variable

    def __init__(self, name, salary):
        self.name = name          # instance variable
        self.salary = salary

    def display(self):            # instance method
        print("Name:", self.name)
        print("Salary:", self.salary)

    @classmethod
    def show_company(cls):        # class method
        print("Company:", cls.company)

    @staticmethod
    def greet():                  # static method
        print("Welcome to the company")


e1 = Employee("Anita", 50000)

e1.display()
Employee.show_company()
Employee.greet()
```

Encapsulation:

- Encapsulation is the process of wrapping data and methods into a single unit called a class.
- It protects data from unauthorized access.
- Encapsulation improves data security and code reliability.
- Access specifiers control data visibility.
- Encapsulation reduces accidental modification of data.

Access Modifiers:

Type	Symbol	Meaning
Public	name	Accessible everywhere
Protected	_name	Accessible within class & child
Private	__name	Accessible only within class

Inheritance

- Inheritance allows one class to acquire the properties and methods of another class.
- The existing class is called the parent class.
- The new class is called the child class.
- Inheritance supports code reuse and faster development.
- It helps build relationships between classes.

Example Code:

```python
class A:
    def show(self):
        print("Parent")

class B(A):
    pass
```

Polymorphism

- Polymorphism means one function name can perform different actions.
- It allows methods to behave differently based on objects.
- Polymorphism increases flexibility and scalability.
- Method overriding is a common example of polymorphism.
- It supports dynamic method execution.

Example Code:

```python
class A:
    def speak(self):
        print("Animal")

class B(A):
    def speak(self):
        print("Dog")
```

Abstraction

- Abstraction hides internal implementation details and shows only essential features.
- It reduces complexity for the user.
- Abstraction is achieved using abstract classes and methods.
- It focuses on what an object does, not how it does it.
- Abstraction improves system security.

Example Code:

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def speed(self):
        pass
```

---

super() Keyword

- The super() keyword is used to access parent class methods and constructors.
- It helps avoid code duplication.
- It improves maintainability of inherited code.

- It ensures parent class initialization.
- It is commonly used in inheritance.

Example Code:

```python
class Parent:
    def __init__(self):
        print("Parent")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child")
```

Magic (Dunder) Methods

- Magic methods start and end with double underscores.
- They provide special behavior to classes.
- They allow operator overloading.
- They improve object representation.
- Examples include __init__, __str__, and __add__.

Mini Project Using OOPS

Project Title: Student Management System

Project Description

This project manages student details such as name, roll number, and marks using OOPS concepts like class, object, constructor, encapsulation, and methods.

Example Code:

```python
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll)
        print("Marks:", self.marks)

    def result(self):
        if self.marks >= 40:
            print("Result: Pass")
        else:
            print("Result: Fail")


s1 = Student("Ravi", 101, 75)
s2 = Student("Anita", 102, 35)

s1.display()
s1.result()

print("--------------")

s2.display()
s2.result()
```